

241-405 Optimization

Topic 08: Graph Optimization3

Minimum Spanning Tree (MST)

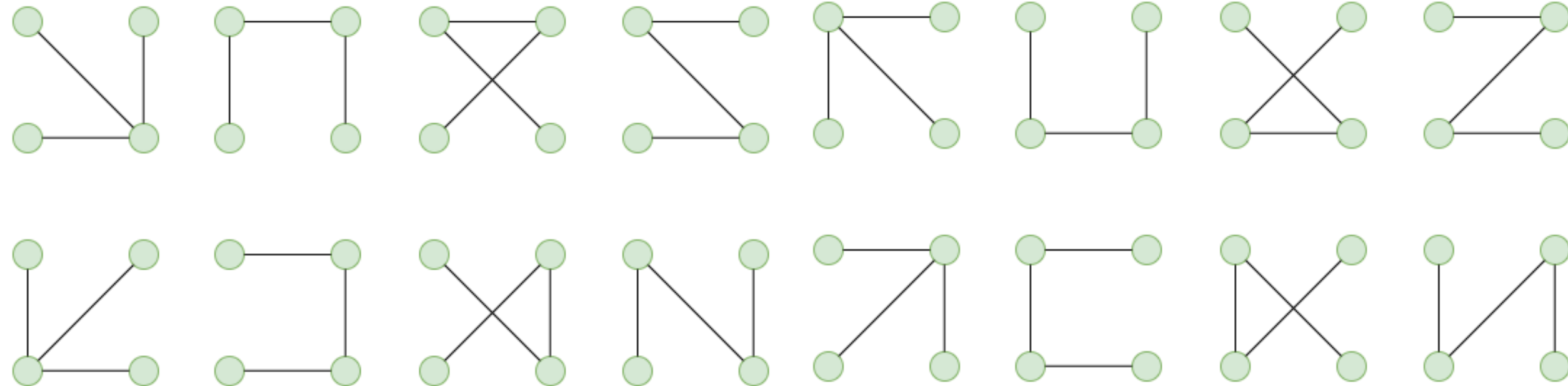
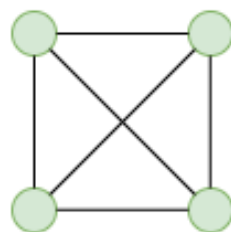
Yossawee Keaomanee

Outline

- บทนำ
- การประยุกต์ใช้ MST
- อัลกอริทึมของครัสคัลและตัวอย่าง
- อัลกอริทึมของพริมและตัวอย่าง
- เปรียบเทียบ Kruskal กับ Prim
- สรุป

Introduction

Graph =



Introduction

Spanning Tree (ST) คืออะไร?

- กราฟย่อยของกราฟแบบไม่มีทิศทางที่เชื่อมต่อกัน
- เชื่อมต่อจุดยอด (Vertices) ทุกจุดเข้าด้วยกัน
- ไม่มีวงจร (Cycle)
- ถ้ามี V node จะมีเส้นเชื่อมจำนวน $V-1$ เส้น

Minimum Spanning Tree (MST) คืออะไร?

- Spanning Tree ที่มี น้ำหนักรวมของเส้นเชื่อมต่ำที่สุดเท่าที่เป็นไปได้

Applications of ST

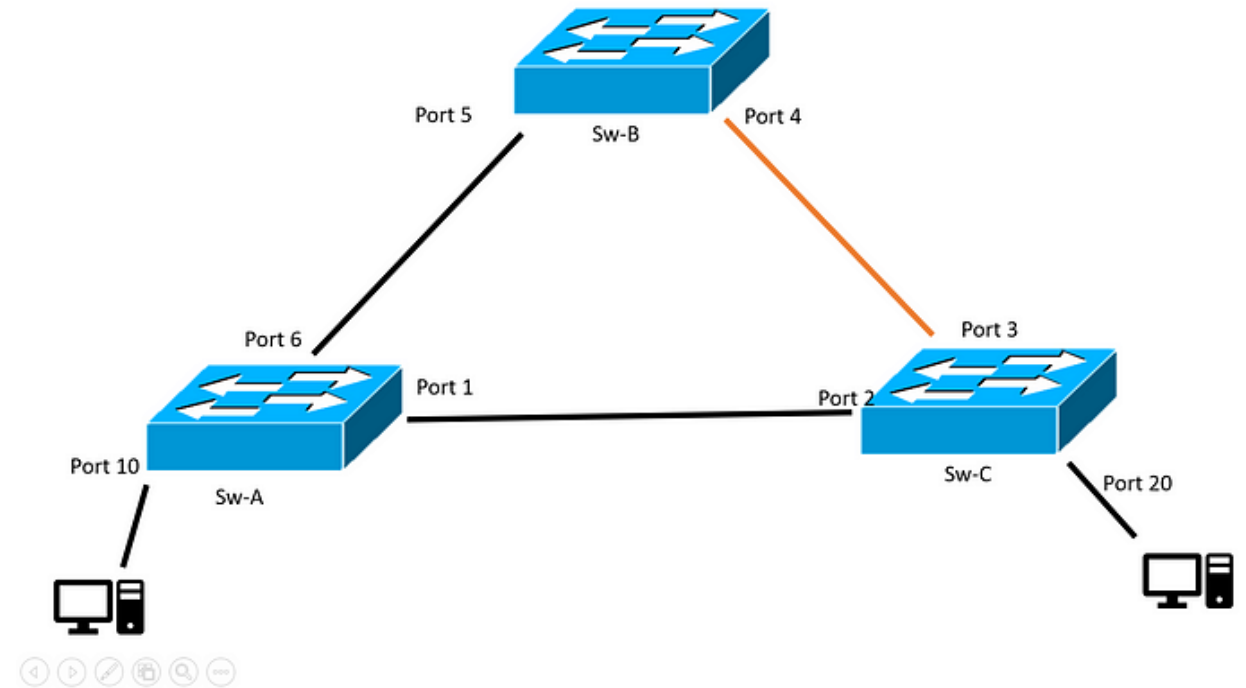
เน้นการสร้างโครงสร้างแบบต้นไม้เพื่อป้องกันการวนลูป (Loop Prevention) และจัดระเบียบการเชื่อมต่อ:

- การออกแบบเครือข่าย:
 - ใช้ในการออกแบบเครือข่ายโทรศัพท์ ระบบไฟฟ้า หรือโครงข่ายถนน โดยมีต้นทุนต่ำที่สุด
- การจัดกลุ่มใน Machine Learning
- การประมาณคำตอบสำหรับปัญหา ปัญหาพนักงานขายเดินทาง (TSP)

Applications of ST

การออกแบบเครือข่าย

- ลดต้นทุนรวมให้ต่ำที่สุด
- ไม่มีการเชื่อมต่อซ้ำซ้อนหรือวงจร
- รับประกันการเชื่อมต่อถึงกัน
- ใช้ทรัพยากรอย่างมีประสิทธิภาพ
- ง่ายต่อการตรวจสอบและแก้ไขปัญหา



<https://mild-tn.medium.com/spanning-tree-protocol-ieee802-1d-f96dfbcaec7e>

Spanning Tree ในเครือข่ายจริง (มาตรฐาน IEEE)

- Spanning Tree Protocol (STP) – *IEEE 802.1D*
- Rapid Spanning Tree Protocol (RSTP) – *IEEE 802.1w*
- Multiple Spanning Tree Protocol (MSTP) – *IEEE 802.1s*

Applications of ST

Spanning Tree ในเครือข่ายจริง (มาตรฐาน IEEE)

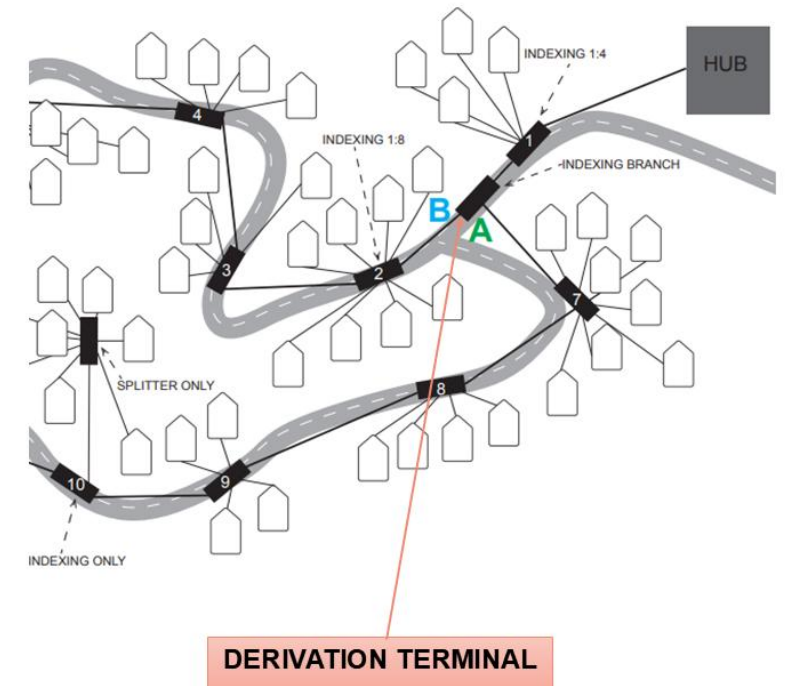
- ป้องกันการเกิดลูปในเครือข่าย
- ป้องกัน Broadcast Storm ด้วยการบล็อกเส้นทางที่ซ้ำซ้อน
- รักษาเสถียรภาพของเครือข่ายด้วยโครงสร้างแบบไม่มีลูป
- รองรับการสำรองเส้นทาง โดยเปิดใช้งานเส้นทางสำรองเมื่อเส้นทางหลักขัดข้อง
- ช่วยให้การตรวจสอบและแก้ไขปัญหาเครือข่ายทำได้ง่ายขึ้น
- รองรับการขยายเครือข่ายขนาดใหญ่ที่มีสวิตช์หลายตัว

Applications of MST

เน้นการลดต้นทุน (Cost), ระยะทาง (Distance) หรือ
ทรัพยากรรวมให้ต่ำที่สุด:

การออกแบบโครงสร้างพื้นฐานและสาธารณูปโภค
(Physical Network Design):

- **Telecommunications:** การลากสายเคเบิล/ใยแก้วนำแสง (Fiber Optic) เพื่อเชื่อมต่อสถานีฐานหรือบ้านเรือนโดยใช้ความยาวสายรวมน้อยที่สุด
- **Power Grid & Piping:** การวางระบบสายไฟแรงสูงหรือท่อประปา/แก๊สธรรมชาติ ให้ครอบคลุมทุกเมืองด้วยงบประมาณก่อสร้างต่ำสุด



Applications of MST

เน้นการลดต้นทุน (Cost), ระยะทาง (Distance) หรือทรัพยากรรวมให้ต่ำที่สุด:

- การออกแบบวงจรรวม (VLSI Circuit Design):
 - การลากเส้นเชื่อมต่อขาลายวงจร (Pins) บนชิปหรือแผงวงจรพิมพ์ (PCB) เพื่อประหยัดพื้นที่ ลบลายวงจรที่ซ้ำซ้อน และลดสัญญาณรบกวน

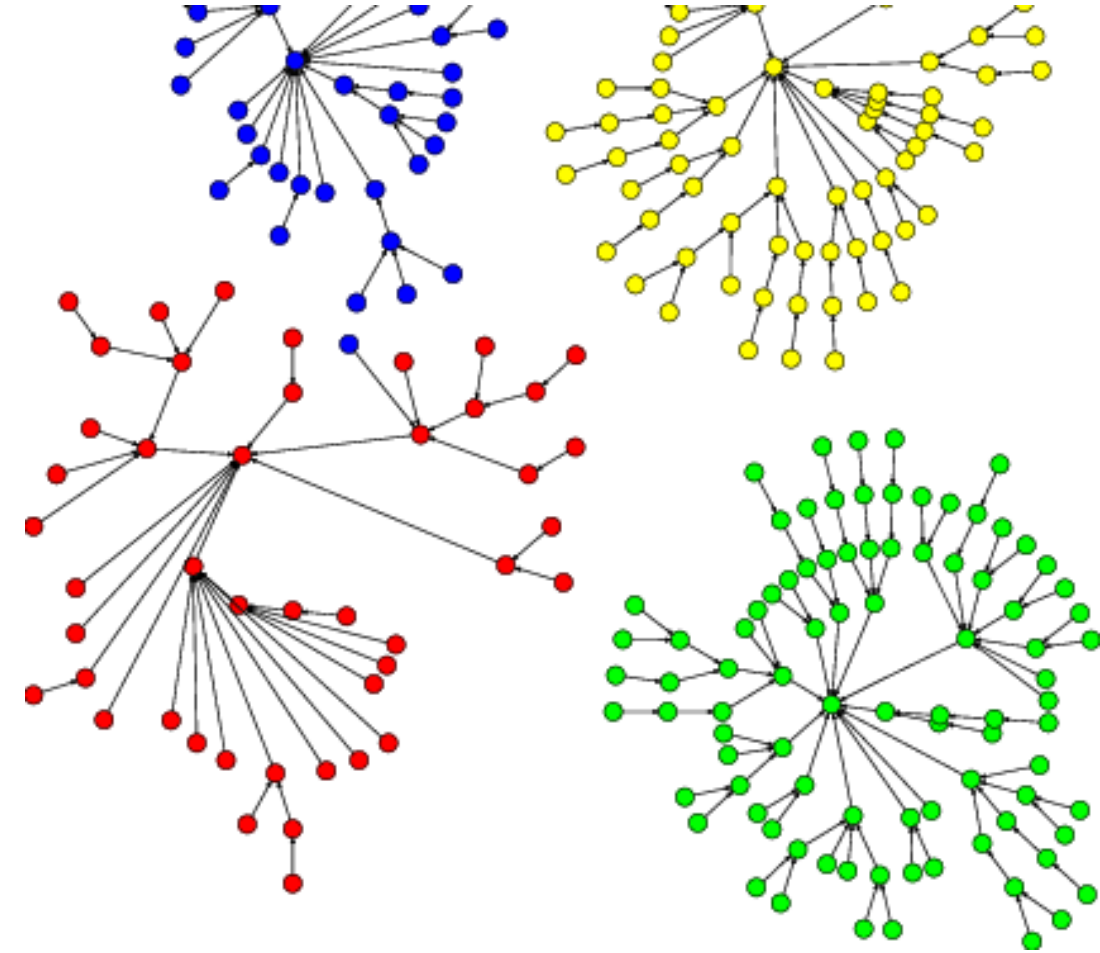


Applications of MST

การจัดกลุ่มข้อมูลด้วย Machine Learning

MST ช่วยในการจัดกลุ่มข้อมูลอย่างไร?

- **ขั้นตอนที่ 1:** สร้าง Minimum Spanning Tree (MST) จากชุดข้อมูล
- **ขั้นตอนที่ 2:** ตัดเส้นเชื่อมที่มีน้ำหนักมากที่สุด ใน MST เพื่อแบ่งกราฟออกเป็นกลุ่ม
- **ผลลัพธ์:** ได้กลุ่มข้อมูลที่มีลักษณะคล้ายกัน โดยมีระยะห่างภายในกลุ่มน้อยที่สุด



Applications of MST

การประมาณค่าคำตอบในปัญหาทาง NP-Hard (Approximation Algorithms):

- TSP (Traveling Salesperson Problem):
ในการหาเส้นทางวนทั้ทุกเมืองด้วยระยะทางที่สั้นที่สุด การหา MST จะถูกนำมาใช้เป็น **Lower Bound** หรือใช้ใน Christofides Algorithm เพื่อหาคำตอบใกล้เคียงที่ดีที่สุด (Approximation Factor 1.5)



MST Problem Definition

- กำหนดให้:

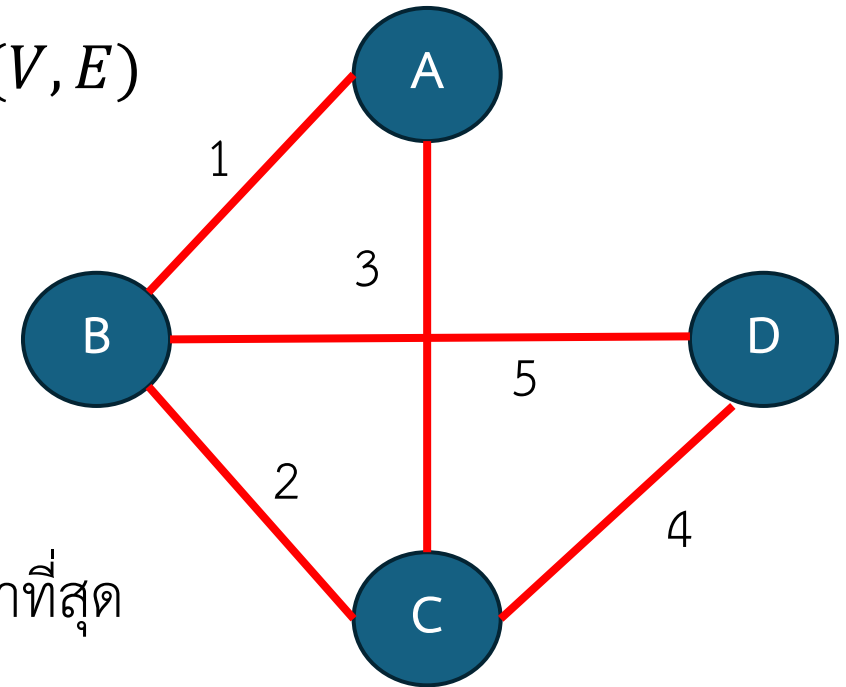
- กราฟถ่วงน้ำหนักแบบไม่มีทิศทางและเชื่อมต่อกัน $G(V, E)$

- ต้องการหา:

- เซตย่อยของเส้นเชื่อม $T \subseteq E$ โดยมีเงื่อนไขว่า
 - T ต้องเป็น ต้นไม้ (Tree)
 - ต้องเชื่อมต่อ จุดยอด (Vertices) ทุกจุด
 - ทำให้ $\sum_{e \in T} \omega(e)$ น้ำหนักรวมของเส้นเชื่อมมีค่าต่ำที่สุด

ผลลัพธ์ :

- มีจำนวนเส้นเชื่อมเท่ากับ $|T| = |V| - 1$ เส้น



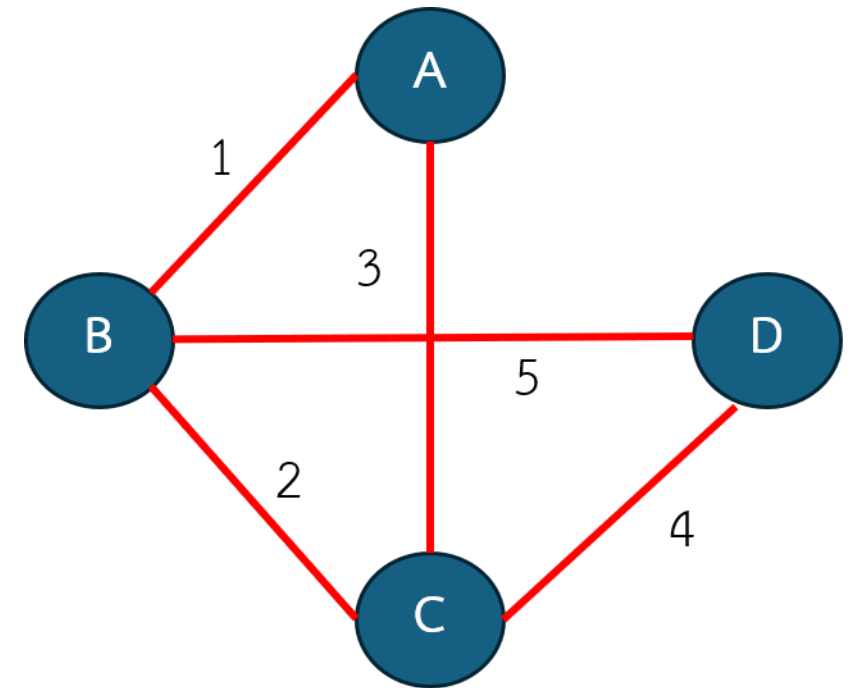
อัลกอริทึมของครัสคัล (Kruskal's Algorithm)

- เรียงเส้นเชื่อมทั้งหมดตาม น้ำหนักจากน้อยไปมาก
- เริ่มต้น MST เป็นเซตว่าง
- พิจารณาเส้นเชื่อมตามลำดับที่เรียงไว้:
 - หากเพิ่มเส้นเชื่อมแล้ว ไม่ทำให้เกิดวงจร $\rightarrow$ เพิ่มเข้า MST
 - หากทำให้เกิดวงจร $\rightarrow$ ข้ามเส้นเชื่อมนั้น
- หยุดเมื่อ MST มีเส้นเชื่อมครบ $|V| - 1$ เส้น.
- โครงสร้างข้อมูลที่ใช้: Disjoint Set (Union-Find) สำหรับตรวจสอบและจัดการกลุ่มของจุดยอด

อัลกอริทึมของครัสคัล (Kruskal's Algorithm)

ตัวอย่าง: เป้าหมายคือการหา Spanning Tree ที่

- เชื่อมจุดยอดทุกจุดในกราฟ เพื่อให้เครือข่ายเชื่อมต่อถึงกัน
- ไม่มีวงจร (loop, cycle) : เพื่อไม่ให้เกิดเส้นทางที่ซ้ำซ้อน
- น้ำหนักรวมของเส้นเชื่อมที่เลือกมีค่าต่ำที่สุด
- สิ่งนี้เรียกว่า Minimum Spanning Tree (MST)



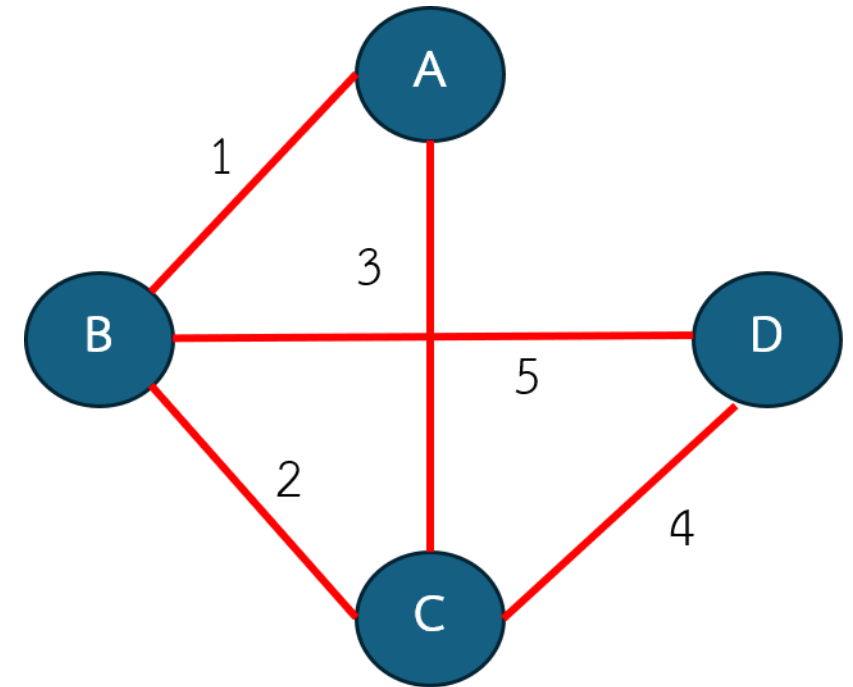
อัลกอริทึมของครัสคัล (Kruskal's Algorithm)

ขั้นตอนที่ 1: เรียงเส้นเชื่อมทั้งหมดตามน้ำหนักจากน้อยไปมาก

Edge	Weight
A-B	1
B-C	2
A-C	3
C-D	4
B-D	5

ขั้นตอนที่ 2: กำหนดให้ MST เป็นเซตว่าง

- $MST = \emptyset$
- ยังไม่มีการเลือกเส้นเชื่อมใด ๆ

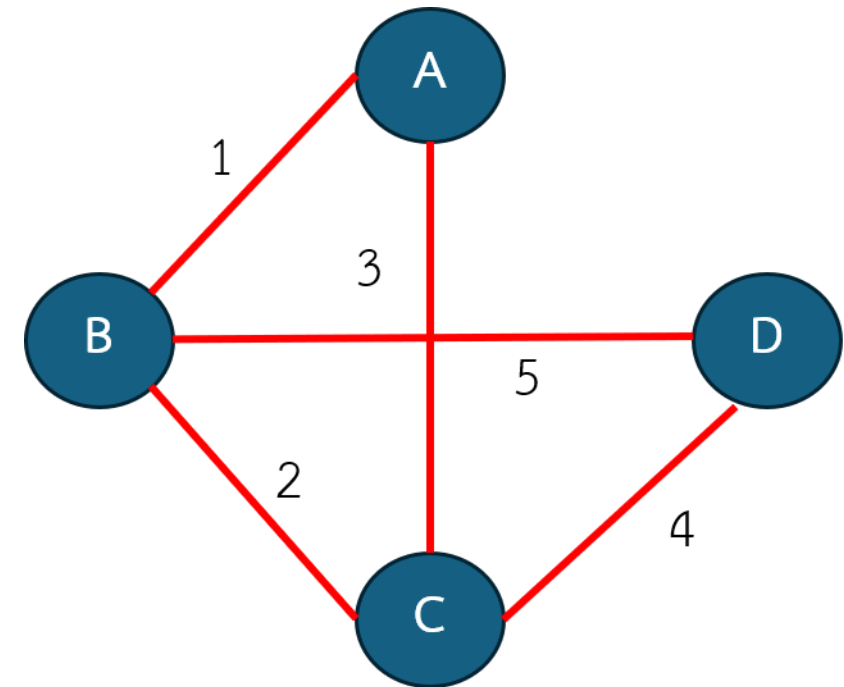


อัลกอริทึมของครัสคัล (Kruskal's Algorithm)

ขั้นตอนที่ 3: พิจารณาเส้นเชื่อมทีละเส้น

- เส้นเชื่อม A-B (น้ำหนัก 1)
 - เพิ่ม A-B เข้า MST เพราะเชื่อม A และ B โดยไม่ทำให้เกิดวงจร
 - **MST = {A-B}**
- เส้นเชื่อม B-C (น้ำหนัก 2)
 - เพิ่ม B-C เพราะเชื่อมจุดยอด C โดยไม่ทำให้เกิดวงจร
 - **MST = {A-B, B-C}**

Edge	Weight
A-B	1
B-C	2
A-C	3
C-D	4
B-D	5



อัลกอริทึมของครัสคัล (Kruskal's Algorithm)

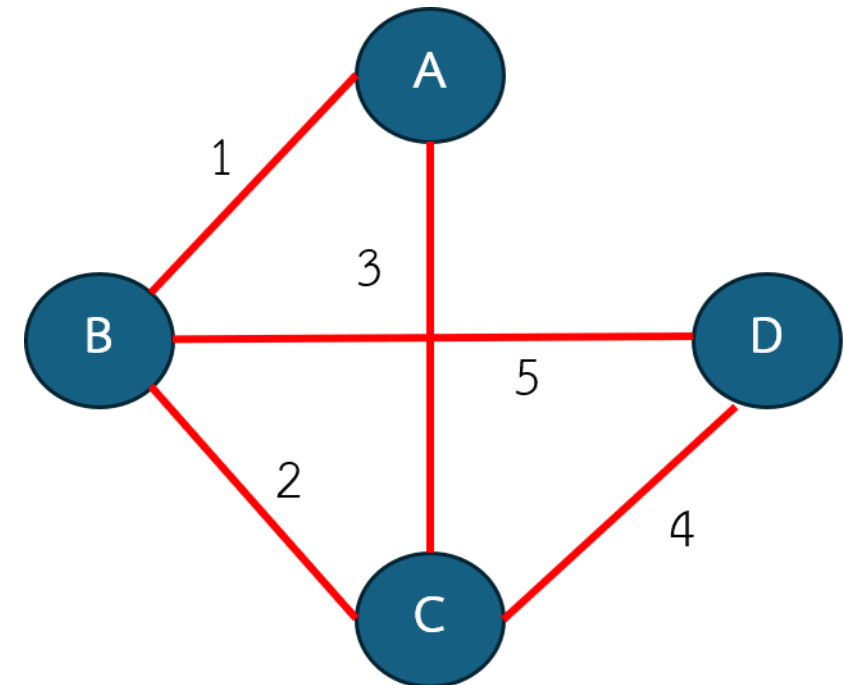
- เส้นเชื่อม A-C (น้ำหนัก 3)

- ไม่เพิ่ม A-C เนื่องจาก A และ C เชื่อมต่อกันอยู่แล้วผ่าน A-B-C
- การเพิ่มเส้นเชื่อมนี้จะทำให้เกิด วงจร (A-B-C-A)
- ข้ามเส้นเชื่อมนี้

- เส้นเชื่อม C-D (น้ำหนัก 4)

- เพิ่ม C-D เพราะเชื่อมจุดยอด D โดยไม่ทำให้เกิดวงจร
- **MST = {A-B, B-C, C-D}**

Edge	Weight
A-B	1
B-C	2
A-C	3
C-D	4
B-D	5



อัลกอริทึมของครัสคัล (Kruskal's Algorithm)

- เส้นเชื่อม B-D (น้ำหนัก 5)

- ข้ามเส้นเชื่อมนี้ เนื่องจาก B และ D เชื่อมต่อกันอยู่แล้วผ่าน B-C-D
- การเพิ่มเส้นเชื่อมนี้จะทำให้เกิด วงจร (B-C-D-B)

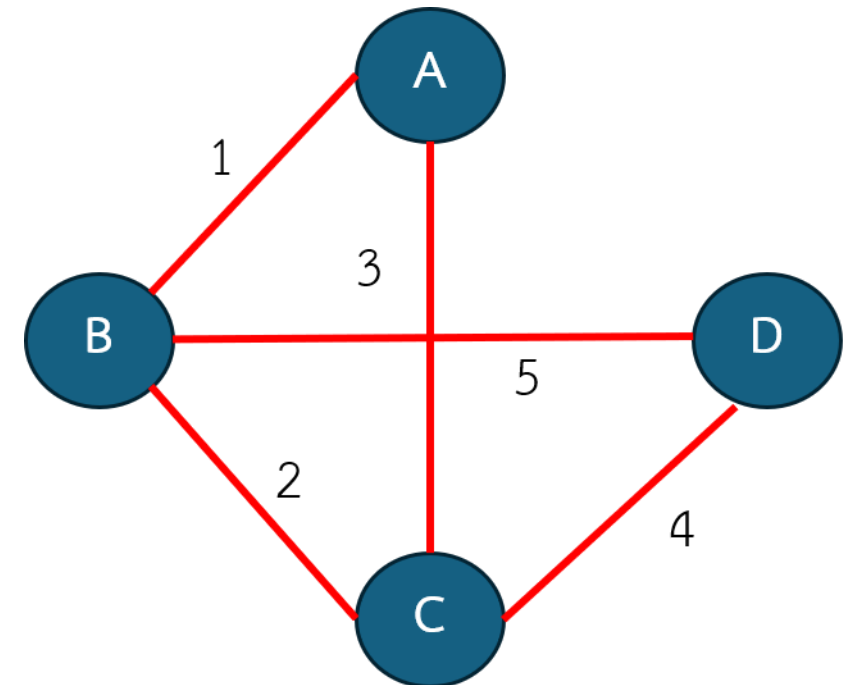
- ผลลัพธ์

- เส้นเชื่อมของ MST: {A-B, B-C, C-D}

- น้ำหนักรวม = $1 + 2 + 4 = 7$

- ดังนั้น ต้นไม้ที่ได้คือ Minimum Spanning Tree (MST)

Edge	Weight
A-B	1
B-C	2
A-C	3
C-D	4
B-D	5



อัลกอริทึมของพริม (Prim's Algorithm)

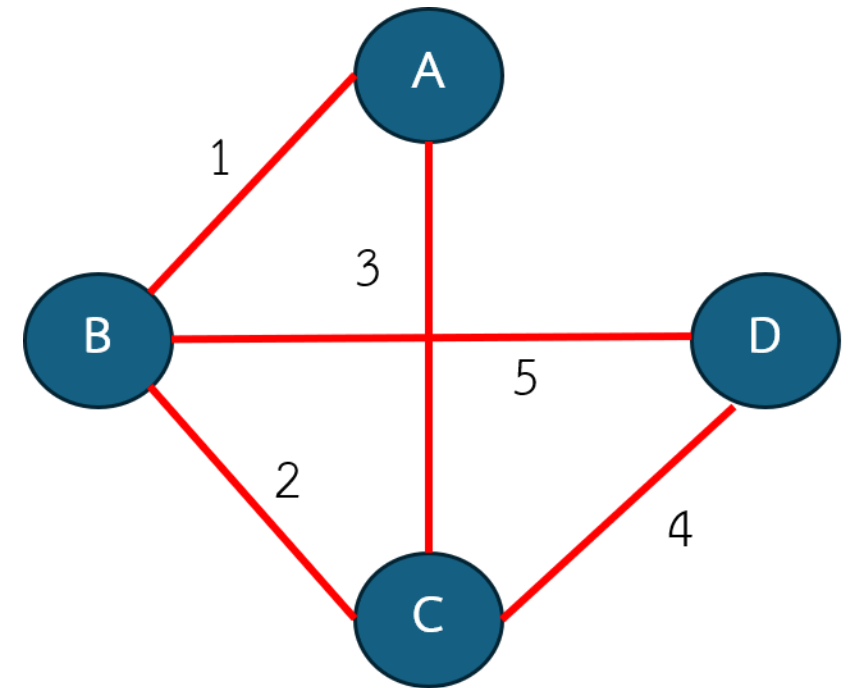
- เริ่มต้นจาก จุดยอดใดก็ได้
- เลือกเส้นเชื่อมที่มี น้ำหนักน้อยที่สุด ซึ่งเชื่อมจากต้นไม้ที่สร้างไว้ไปยังจุดยอดใหม่
- ทำซ้ำจนกระทั่ง จุดยอดทุกจุดถูกเพิ่มเข้าไปในต้นไม้

โครงสร้างข้อมูลที่ใช้: Min-Heap (Priority Queue)

อัลกอริทึมของพริม (Prim's Algorithm)

กราฟ: ใช้กราฟถ่วงน้ำหนักเดียวกันกับตัวอย่างของ Kruskal

- ขั้นตอนการทำงาน (เริ่มต้นที่ A):
- เลือก A-B (น้ำหนัก 1)
- เลือก B-C (น้ำหนัก 2)
- เลือก C-D (น้ำหนัก 4)

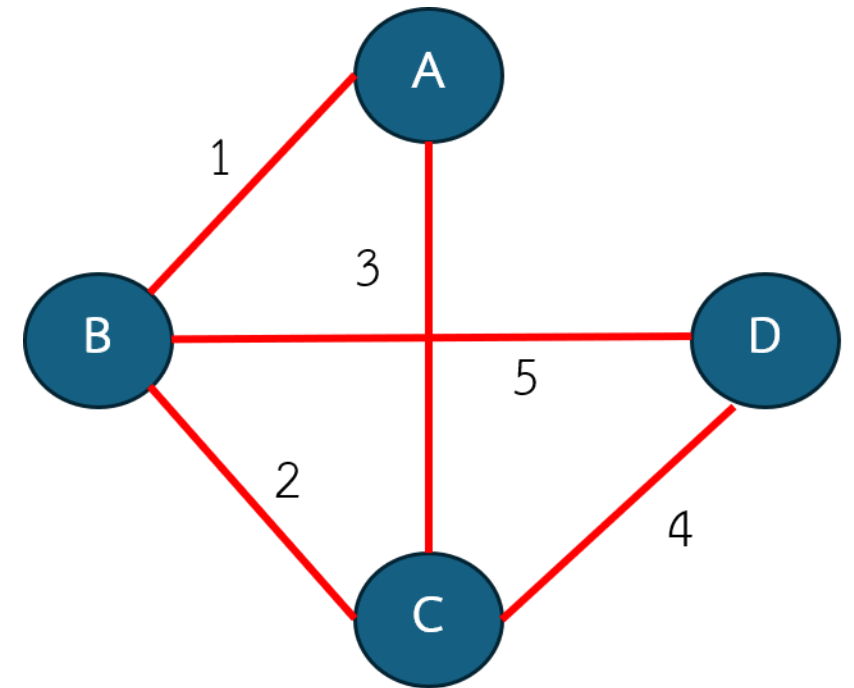


อัลกอริทึมของพริม (Prim's Algorithm)

เป้าหมายคือการสร้าง Minimum Spanning Tree (MST):

- เชื่อมต่อจุดยอด (โหนด) ทุกจุดเข้าด้วยกัน
- ไม่มีวงจรหรือการเชื่อมต่อที่ซ้ำซ้อน
- ทำให้น้ำหนักรวมของเส้นเชื่อมที่เลือกมีค่าต่ำที่สุด

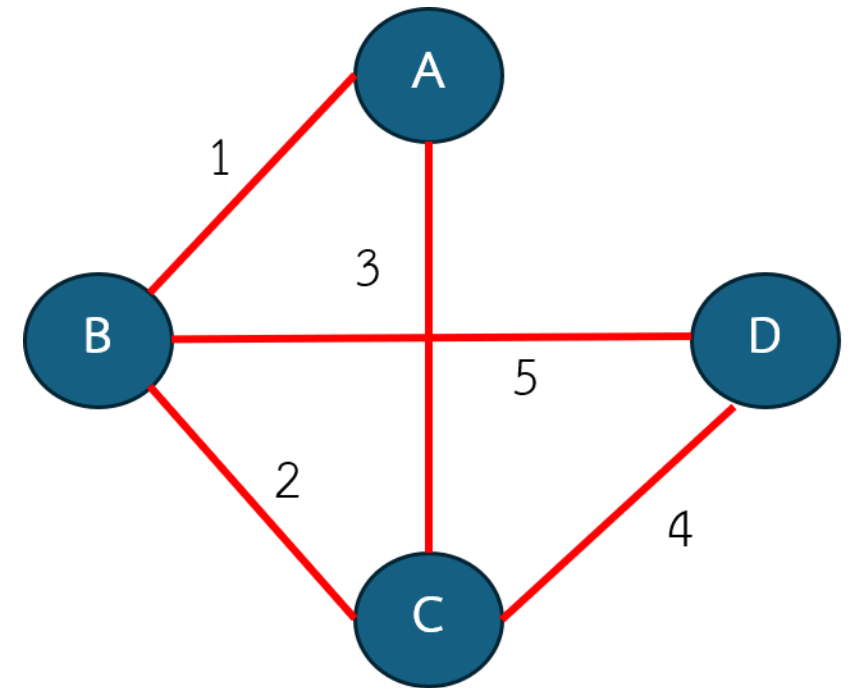
แตกต่างจาก Kruskal : Prim จะเริ่มต้นจาก จุดยอดเพียงจุดเดียว แล้วค่อย ๆ ขยาย tree โดยเพิ่มจุดยอดที่อยู่ใกล้ที่สุดในแต่ละขั้นตอน



อัลกอริทึมของพริม (Prim's Algorithm)

ขั้นตอนที่ 1: เริ่มต้นจากจุดยอดใดก็ได้ (เช่น A)

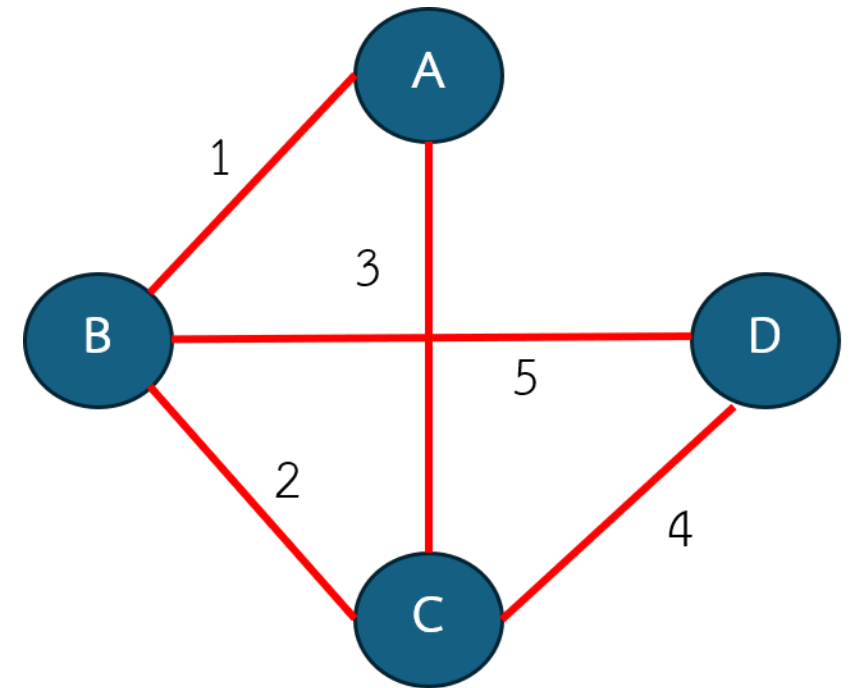
- กำหนดให้ A เป็นจุดเริ่มต้นของ MST
- **MST = {A}**
- จุดยอดที่อยู่ใน MST: A
- จุดยอดที่เหลือ: B, C, D



อัลกอริทึมของพริม (Prim's Algorithm)

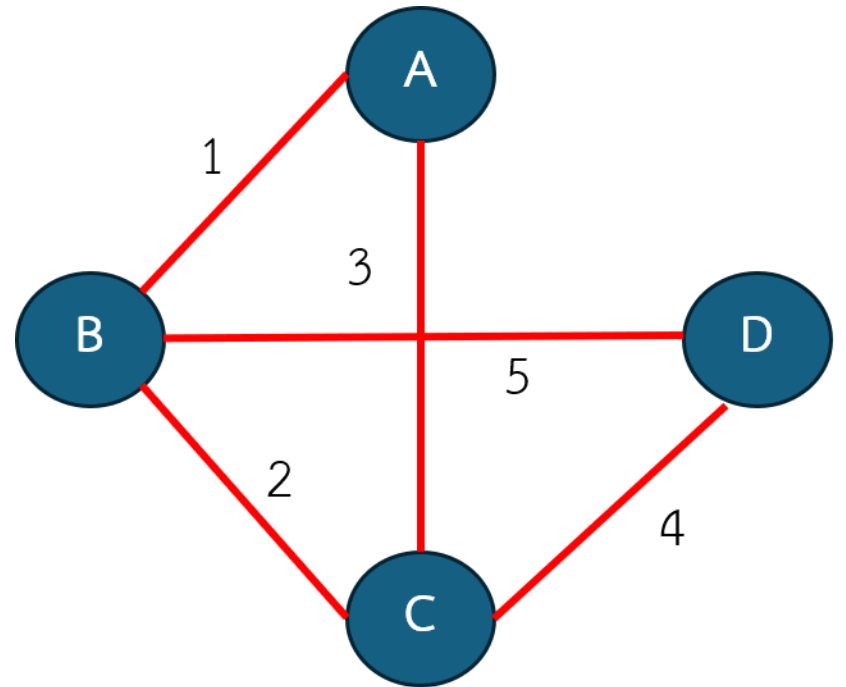
ขั้นตอนที่ 2: เพิ่มจุดยอดที่อยู่ใกล้ที่สุดเข้า MST

- พิจารณาเส้นเชื่อมที่ออกจาก A
 - A-B (น้ำหนัก 1)
 - A-C (น้ำหนัก 3)
- เลือก A-B (น้ำหนัก 1) เพราะมีน้ำหนักน้อยที่สุด
- เพิ่ม B เข้าไปใน MST
- **MST = {A, B}**
- เส้นเชื่อมที่เลือก: A-B



อัลกอริทึมของพริม (Prim's Algorithm)

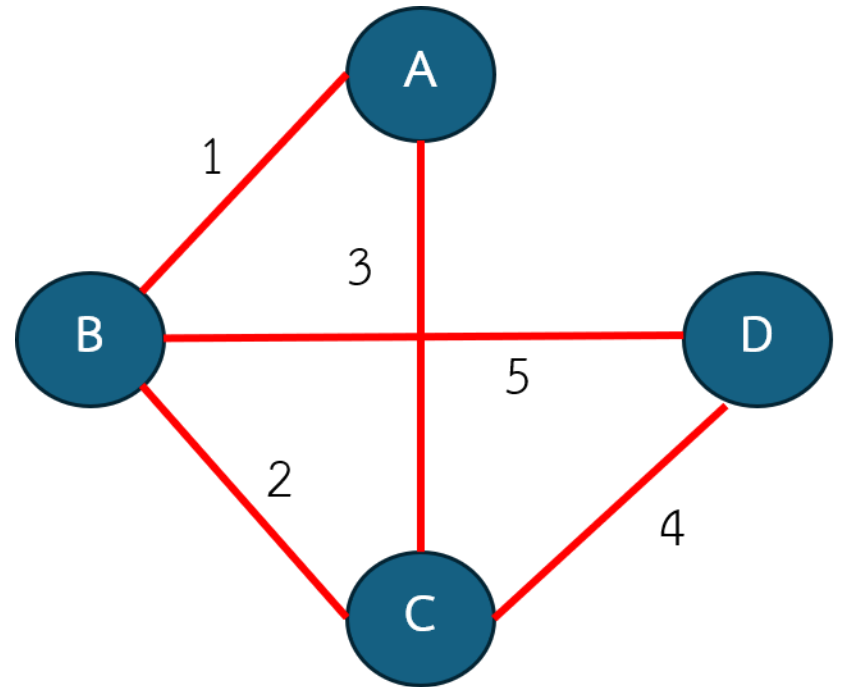
- จากจุดยอด **A** และ **B** ที่อยู่ใน MST ให้พิจารณาเส้นเชื่อมไปยังจุดยอดที่ยังไม่อยู่ใน MST
 - B-C (น้ำหนัก 2)
 - A-C (น้ำหนัก 3)
 - B-D (น้ำหนัก 5)
- เลือก B-C (น้ำหนัก 2) เพราะมีน้ำหนักน้อยที่สุด
- เพิ่ม C เข้าไปใน MST
- **MST = {A, B, C}**
- เส้นเชื่อมที่เลือก: A-B, B-C



อัลกอริทึมของพริม (Prim's Algorithm)

ขั้นตอนที่ 4: เพิ่มจุดยอดที่อยู่ใกล้ที่สุดเข้า MST

- จากจุดยอด A, B และ C ที่อยู่ใน MST ให้พิจารณาเส้นเชื่อมไปยังจุดยอดที่ยังไม่อยู่ใน MST
 - C-D (น้ำหนัก 4)
 - B-D (น้ำหนัก 5)
- เลือก C-D (น้ำหนัก 4) เพราะมีน้ำหนักน้อยที่สุด
- เพิ่ม D เข้าไปใน MST
- MST = {A, B, C, D}
- เส้นเชื่อมที่เลือก: A-B, B-C, C-D



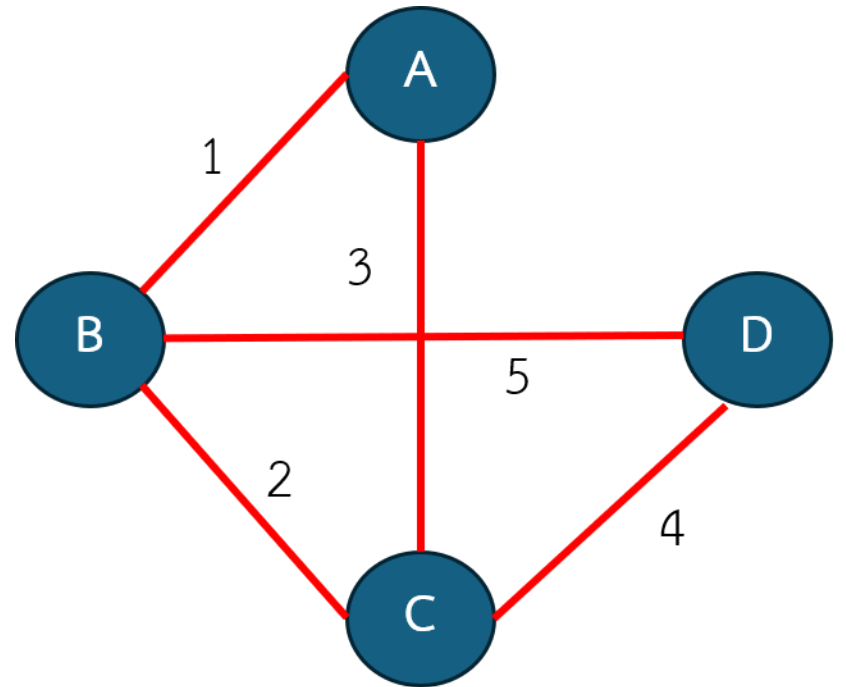
อัลกอริทึมของพริม (Prim's Algorithm)

ผลลัพธ์

- เส้นเชื่อมของ MST: A-B, B-C, C-D
- น้ำหนักรวม = $1 + 2 + 4 = 7$

ประเด็นสำคัญ

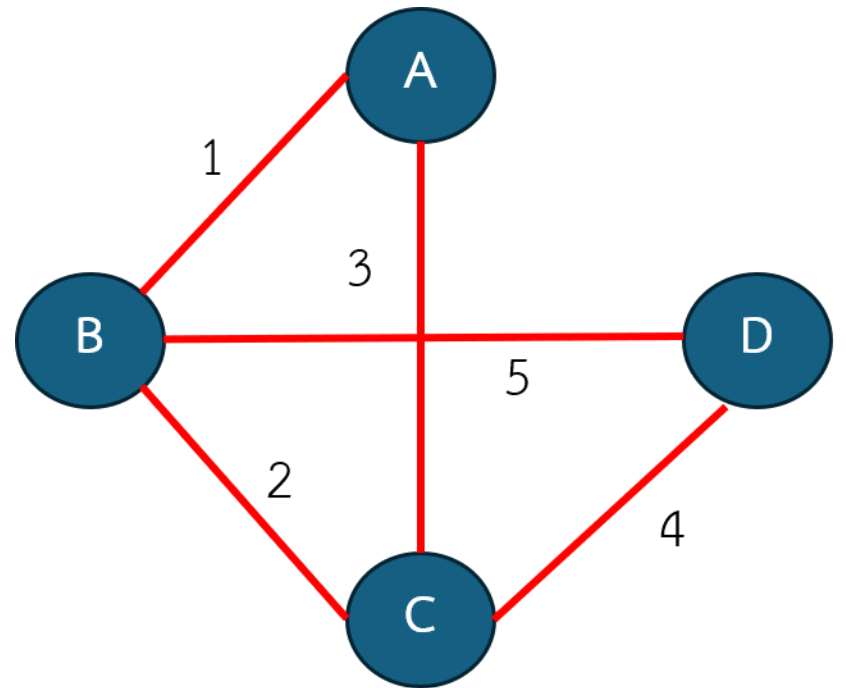
- MST เชื่อมต่อจุดยอดทั้ง 4 จุดเข้าด้วยกัน
- จำนวนเส้นเชื่อมทั้งหมด = $V - 1 = 3$ เส้น
- น้ำหนักรวมต่ำที่สุด = 7
- ไม่มีวงจร (Cycle)



อัลกอริทึมของพริม (Prim's Algorithm)

ขั้นตอนที่ 1: เริ่มต้นจากจุด B

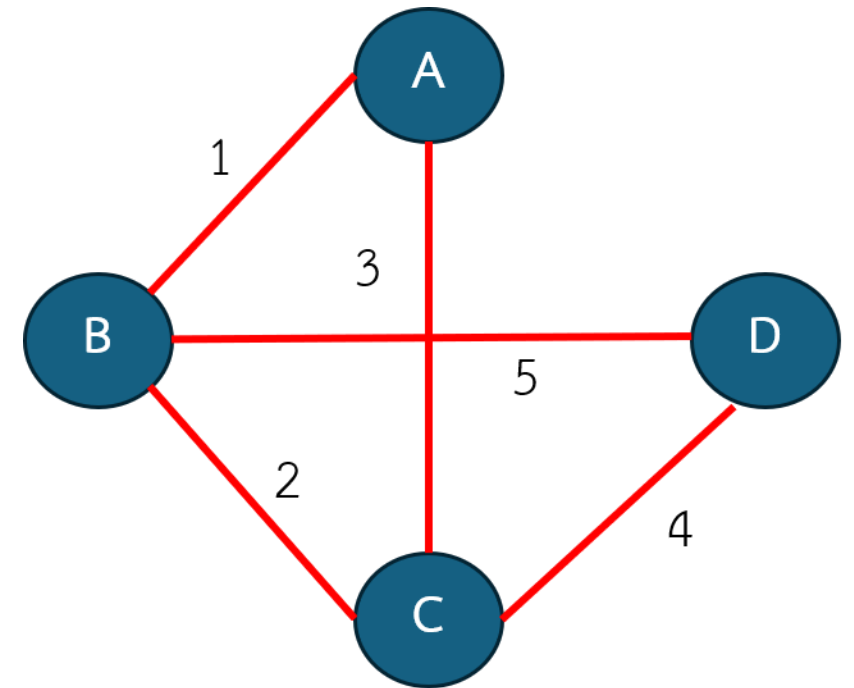
- กำหนดให้ B เป็นจุดเริ่มต้นของ MST
- **MST = {B}**
- จุดยอดที่อยู่ใน MST: B
- จุดยอดที่เหลือ: A, C, D



อัลกอริทึมของพริม (Prim's Algorithm)

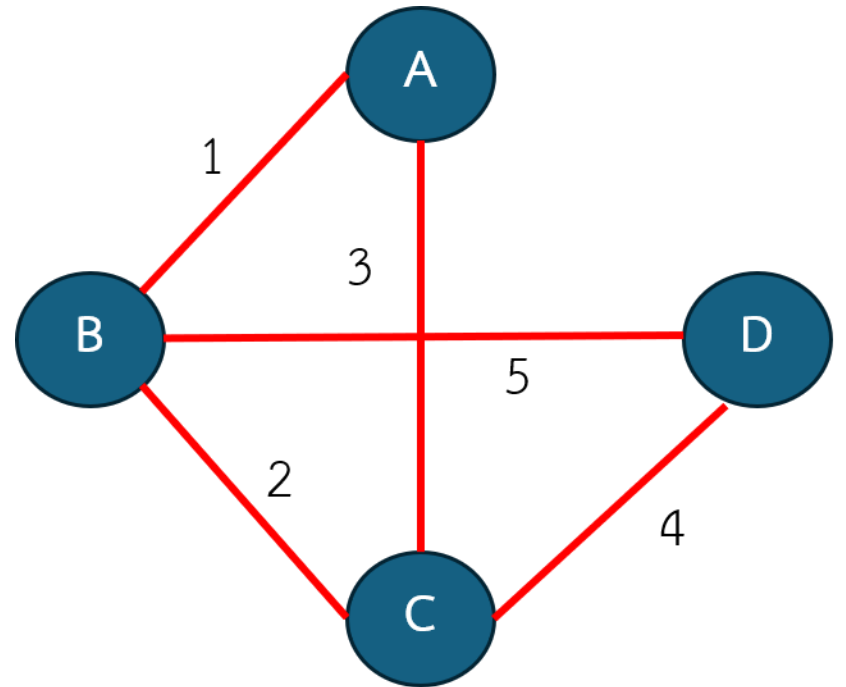
ขั้นตอนที่ 2: เพิ่มจุดยอดที่อยู่ใกล้ที่สุดเข้า MST

- พิจารณาเส้นเชื่อมที่ออกจาก B
 - B-A (น้ำหนัก 1)
 - B-C (น้ำหนัก 2)
- เลือก B-A (น้ำหนัก 1) เพราะมีน้ำหนักน้อยที่สุด
- เพิ่ม A เข้าไปใน MST
- MST = {B,A}
- เส้นเชื่อมที่เลือก: B-A



อัลกอริทึมของพริม (Prim's Algorithm)

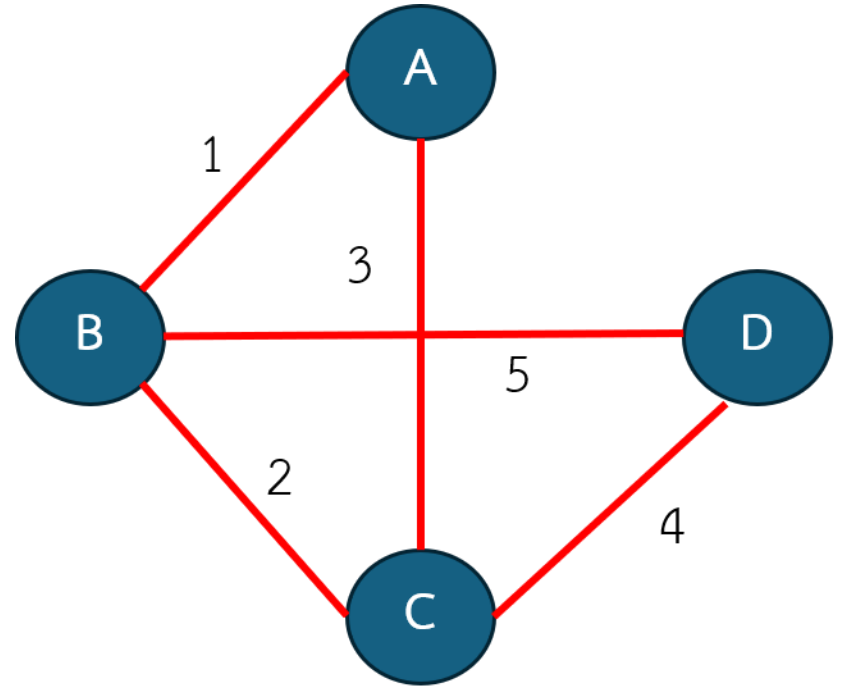
- จากจุดยอด B และ A ที่อยู่ใน MST ให้พิจารณาเส้นเชื่อมไปยังจุดยอดที่ยังไม่อยู่ใน MST
 - B-C (น้ำหนัก 2)
 - A-C (น้ำหนัก 3)
 - B-D (น้ำหนัก 5)
- เลือก B-C (น้ำหนัก 2) เพราะมีน้ำหนักน้อยที่สุด
- เพิ่ม C เข้าไปใน MST
- MST = {B, A, C}
- เส้นเชื่อมที่เลือก: B-A, B-C



อัลกอริทึมของพริม (Prim's Algorithm)

ขั้นตอนที่ 4: เพิ่มจุดยอดที่อยู่ใกล้ที่สุดเข้า MST

- จากจุดยอด B, A และ C ที่อยู่ใน MST ให้พิจารณาเส้นเชื่อมไปยังจุดยอดที่ยังไม่อยู่ใน MST
 - C-D (น้ำหนัก 4)
 - B-D (น้ำหนัก 5)
- เลือก C-D (น้ำหนัก 4) เพราะมีน้ำหนักน้อยที่สุด
- เพิ่ม D เข้าไปใน MST
- MST = {B, A, C, D}
- เส้นเชื่อมที่เลือก: B-A, B-C, C-D



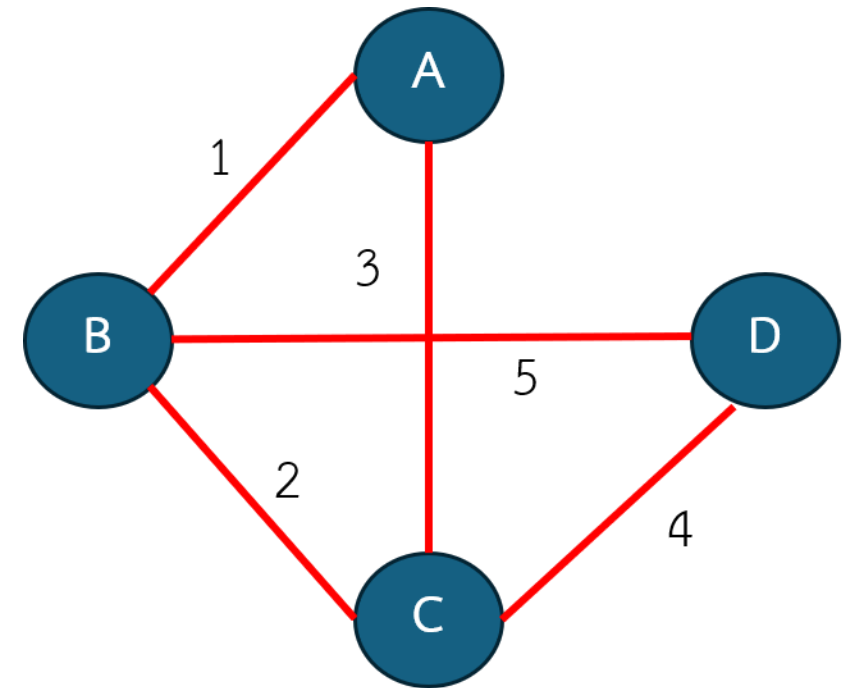
อัลกอริทึมของพริม (Prim's Algorithm)

ผลลัพธ์

- เส้นเชื่อมของ MST: B-A, B-C, C-D
- น้ำหนักรวม = $1 + 2 + 4 = 7$

ประเด็นสำคัญ

- จำนวนเส้นเชื่อมทั้งหมด = $V - 1 = 3$ เส้น
- น้ำหนักรวมต่ำที่สุด = 7 และ ไม่มีวงจร (Cycle)



Kruskal vs Prim

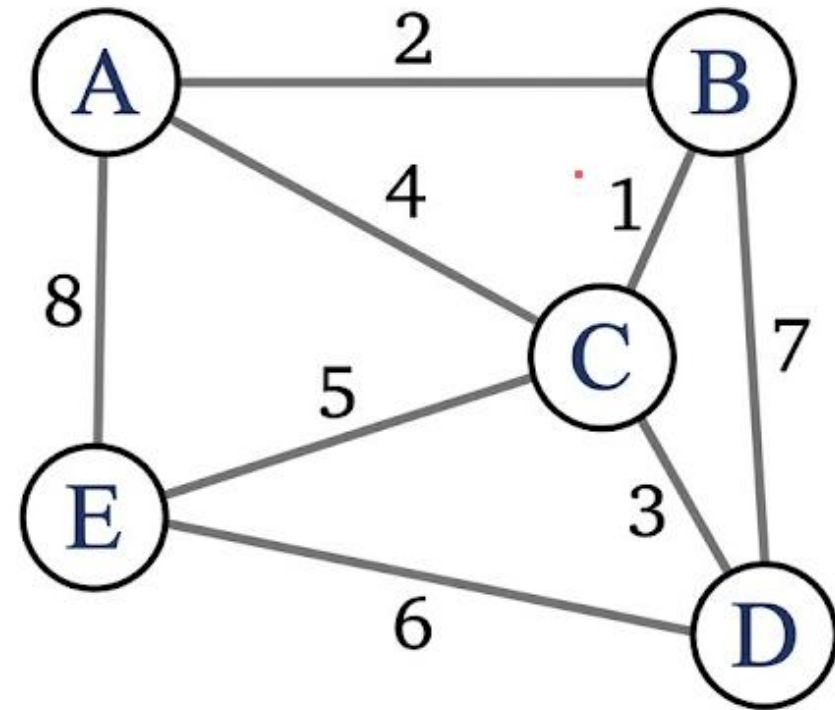
หัวข้อ	Kruskal	Prim
แนวคิดหลัก	เลือก Edge ที่มีน้ำหนักน้อยที่สุด	เลือก Vertex ที่เชื่อมต่อแล้ว ขยายออกไป
วิธีเริ่มต้น	ไม่จำเป็นต้องเลือก Vertex เริ่มต้น	ต้องเลือก Starting Vertex
การเลือก	เลือก Edge ที่เล็กที่สุดทั่วทั้งกราฟ	เลือก Edge ที่เล็กที่สุดจากกลุ่ม Vertex ที่มีอยู่

Kruskal vs Prim

หัวข้อ	Kruskal	Prim
การเกิด Cycle	ต้องตรวจสอบและป้องกัน Cycle	โดยธรรมชาติไม่สร้าง Cycle
โครงสร้าง ระหว่างทำ	อาจเกิดหลาย Connected Components	มีต้นไม้เดียวและค่อย ๆ ขยาย
เหมาะกับ	Sparse Graph	Dense Graph

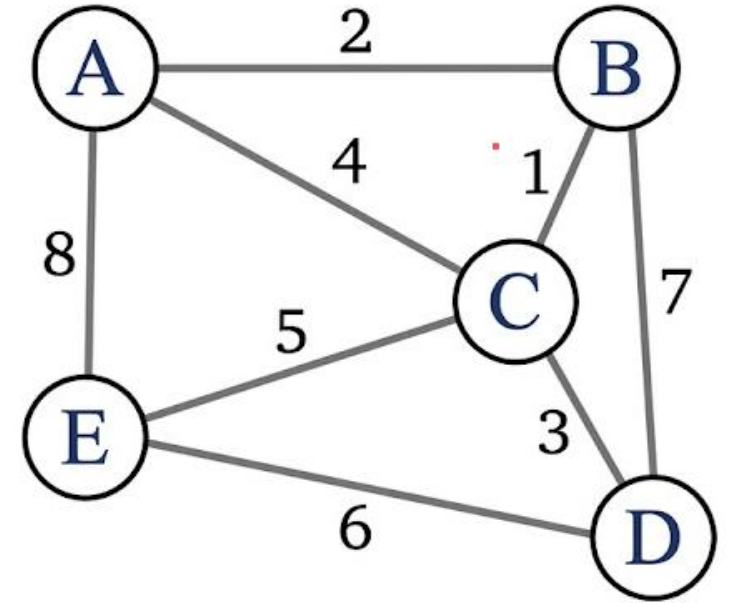
Exercise

- ใช้ Kruskal และ Prim หา MST ของกราฟดังรูป



Exercise

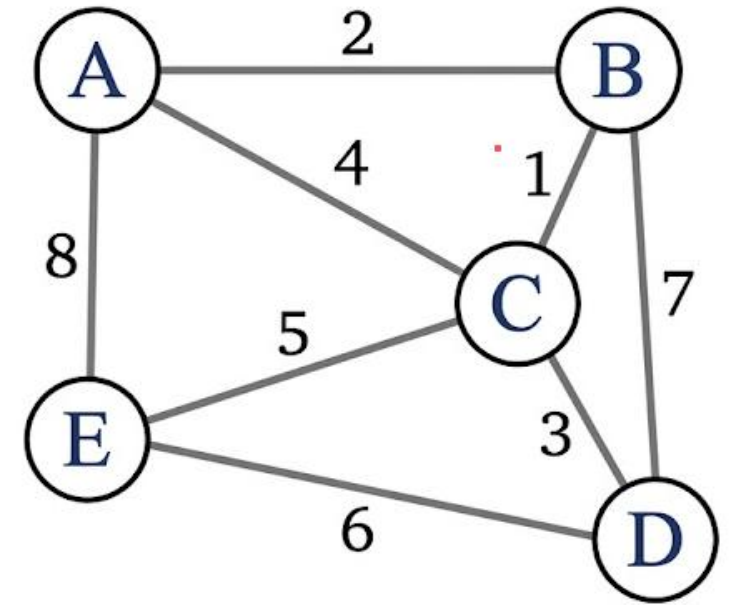
- Kruskal's Algorithm (เรียงน้ำหนักเส้นเชื่อมจากน้อยไปมาก แล้วเลือกโดยไม่ให้เกิด Cycle)
- พิจารณาเลือกทีละเส้นเชื่อม (กราฟ 5 โหนด ต้องใช้ 4 เส้นเชื่อม)



Edge	Weight
(B, C)	1
(A, B)	2
(C, D)	3
(A, C)	4
(C, E)	5
(D, E)	6
(B, D)	7
(A, E)	8

Exercise

- เลือก (B, C) = 1 (ผ่าน)
- เลือก (A, B) = 2 (ผ่าน)
- เลือก (C, D) = 3 (ผ่าน)
- พิจารณา (A, C) = 4 → **ข้าม** (เพราะถ้าเลือกจะเกิดวงรอบ $A - B - C - A$)
- เลือก (C, E) = 5 (ผ่าน)
- **ครบ 4 เส้นเชื่อมแล้ว หยุดการทำงาน**
- น้ำหนักรวมของ MST: $1 + 2 + 3 + 5 = 11$

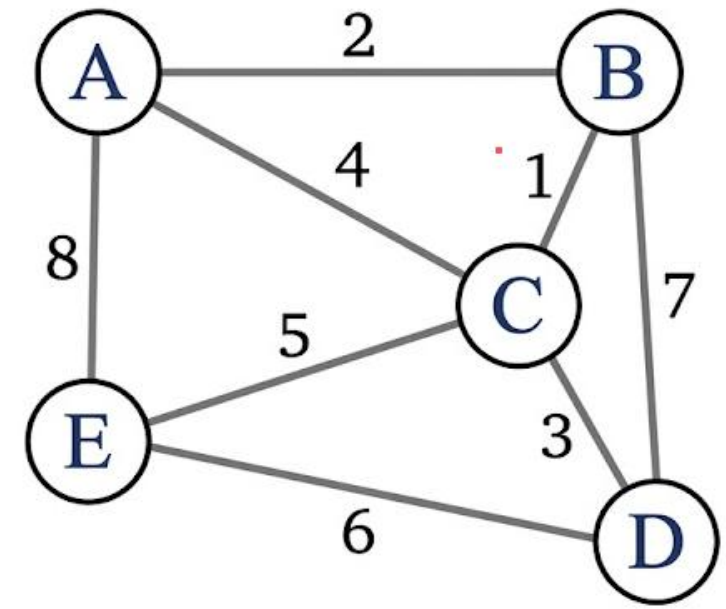


Edge	Weight
(B, C)	1
(A, B)	2
(C, D)	3
(A, C)	4
(C, E)	5
(D, E)	6
(B, D)	7
(A, E)	8

Exercise

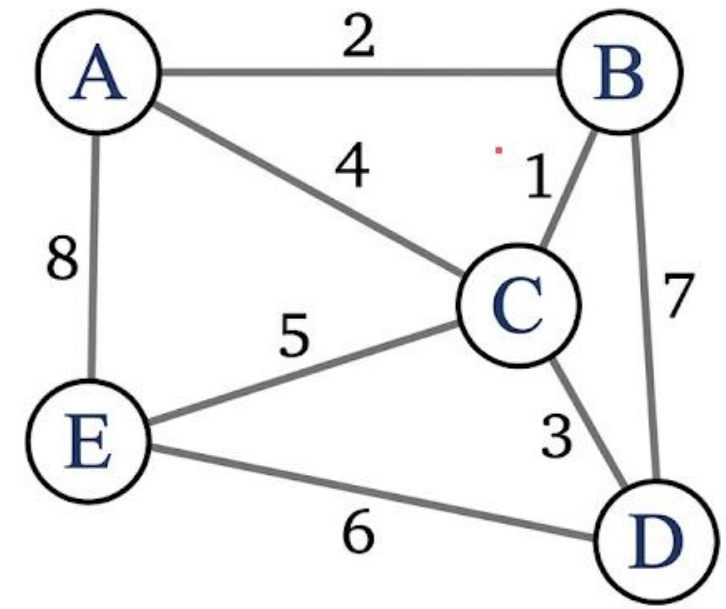
Prim's Algorithm (เริ่มจากโหนด A แล้วขยายไปยังโหนดที่ใกล้เคียงที่สุด)

- เริ่มที่โหนด A
- เส้นเชื่อมจาก A คือ $(A,B)=2$, $(A,C)=4$, $(A,E)=8 \rightarrow$ เลือก $(A, B) = 2$
- $MST = \{A, B\}$
- เส้นเชื่อมที่เลือก: A-B



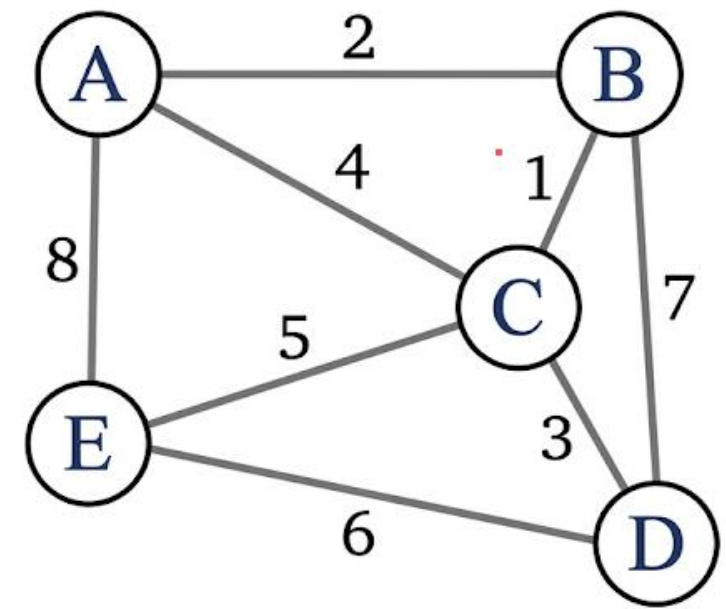
Exercise

- เส้นเชื่อมจาก $\{A, B\}$ ออกไปภายนอก คือ
 - $(A,C)=4$
 - $(B,C)=1$
 - $(B,D)=7$
 - $(A,E)=8$
- เลือก $(B, C) = 1$
- $MST = \{A, B, C\}$
- เส้นเชื่อมที่เลือก: $A-B, B-C$



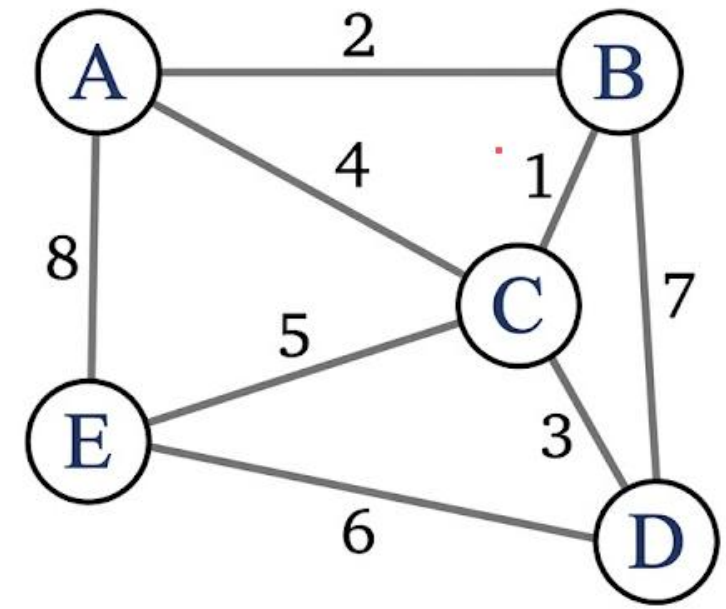
Exercise

- เส้นเชื่อมจาก $\{A, B, C\}$ ออกไปภายนอก คือ
 - $(C,D)=3$
 - $(C,E)=5$
 - $(B,D)=7$
 - $(A,E)=8$
- เลือก $(C, D) = 3$
- $MST = \{A, B, C, D\}$
- เส้นเชื่อมที่เลือก: $A-B, B-C, C-D$



Exercise

- เส้นเชื่อมจาก $\{A, B, C, D\}$ ออกไปภายนอกที่น้อยที่สุด คือ $(C, E) = 5$
- $MST = \{A, B, C, D, E\}$
- เส้นเชื่อมที่เลือก: $A-B, B-C, C-D, C-E$
- เส้นเชื่อมที่อยู่ใน MST : $(A, B), (B, C), (C, D), (C, E)$
- น้ำหนักรวมของ MST : $2 + 1 + 3 + 5 = 11$



Kruskal vs Prim

คุณลักษณะ	Kruskal	Prim
แนวทาง	แบบพิจารณาเส้นเชื่อม (Edge-based)	แบบพิจารณาจุดยอด (Vertex-based)
โครงสร้างข้อมูล	Union-Find	Min-Heap
เหมาะสำหรับ	กราฟเบาบาง (Sparse Graph)	กราฟหนาแน่น (Dense Graph)
ต้องเรียงลำดับเส้นเชื่อมหรือไม่	ใช่	ไม่จำเป็น
เริ่มต้นจาก	เส้นเชื่อมใดก็ได้	จุดยอดใดก็ได้
ความซับซ้อนในการคำนวณ	$O(E \log E)$	$O(E + V \log V)$

V = Number of vertices

E = Number of edges

Summary

- MST เชื่อมต่อจุดยอดทุกจุดด้วย **ต้นทุนรวมต่ำที่สุด**
- Kruskal และ Prim เป็นอัลกอริทึมแบบ **Greedy** สำหรับการหา MST
- เลือกใช้ **Kruskal** สำหรับกราฟเบาบาง (Sparse Graph)
- เลือกใช้ **Prim** สำหรับกราฟหนาแน่น (Dense Graph)

ความแตกต่างระหว่าง MST กับ Shortest path

- MST (Minimum Spanning Tree) คือการเลือกเส้นเชื่อมเพื่อ เชื่อมทุก โหนด (Nodes) ในกราฟเข้าด้วยกัน ให้กลายเป็นต้นไม้ที่ไม่มีวงรอบ โดยใช้ ผลรวมน้ำหนักของทั้งระบบน้อยที่สุด
- Shortest Path คือการหา เส้นทางที่สั้นที่สุดระหว่าง 2 โหนด เฉพาะเจาะจง (เช่น จาก Source ไป Destination) โดยสนใจแค่น้ำหนักรวม ของเส้นทางนั้นๆ เท่านั้น

เปรียบเทียบการใช้งานระหว่าง MST กับ Shortest path

- **MST:** เหมาะกับงานวางโครงสร้างพื้นฐาน เช่น การเดินสายไฟ หรือสายแลน ให้ครอบคลุมทุกอาคารด้วยความยาวสายรวมที่สั้นที่สุด
- **Shortest Path:** เหมาะกับระบบนำทาง (เช่น Google Maps) หรือ Routing ในเครือข่าย เพื่อหาเส้นทางเดินทางจากจุด A ไป B ที่เร็วที่สุด

Shortest Path จำเป็นต้องหา MST ก่อนหรือไม่

- **ไม่จำเป็น** ทั้งสองอย่างใช้อัลกอริทึมที่แยกออกจากกันโดยสิ้นเชิง การหา Shortest Path (เช่น Dijkstra หรือ A*) สามารถคำนวณจากกราฟเต็มได้ทันที ไม่ต้องผ่านการหา MST ก่อน